

3 phases:

1) What is Natural Language Processing?

A brief intro to natural language processing.

2) Applications;

Some applications of NLP.

3) Approaches;

Some approaches to doing NLP.

What is NLP?

NLP(Natural Language Processing) is a part of Artificial Intelligence that helps computers understand, read, write, and talk like humans.

In short, NLP allows machines to process human language like; english, urdu, hindi, etc. instead of just 0 or 1.

You type "Weather in Karachi" in Google. Google understands your text and shows you the correct answer. That's NLP.

You say, "Hey Alexa, play Arijit songs" Alexa listens, understands your sentence, and plays the music. That's NLP working behind the scenes.

Again just for the sake of your machine to understand the language we use NLP. And this language is different from the computer languages like python, java, C etc.

Need of NLP:

Humans talk in language like Hindi, English, Urdu. But computers only understand number(0 and 1). To bridge this gap, we need NLP so that computers can read and understand our language.

Real world Applications:

Chatbots & Virtual Assistants:

Example: ChatGPT, Alexa, Siri, Google Assistant

- * NLP allows these tools to understand what you're saying or typing.
- * They recognize your intent and give a relevant response.
- * This involves speech recognition, text understanding, and response generation.

Email Spam Filtering:

Example: Gmail Spam Detection

- * NLP scans the content of emails for suspicious or spammy phrases like "You won a lottery!"
- * It helps filter out spam and send important emails to the inbox.

Sentiment Analysis:

Example: Twitter Analysis, Amazon Reviews, Customer Feedback

- * Companies use NLP to detect whether people are feeling positive, negative, or neutral about their product.

- * Very useful for brand monitoring and public opinion analysis.

Now there are more and more applications in use the list goes on and on and the use is growing more and more.

Approaches of NLP:

1) Rule-based approach (old school NLP)

Early method of solving NLP using handwritten rules and grammar logic.

- * You write rules manually to process and understand language.

- * These rules can define grammar, spelling, corrections, sentence structure, etc.

Example: A rule like;

If sentence contains 'not good', mark it as negative sentiment.

Used for:

grammar checkers, spell check, simple keyword-based chatbots

Limitation:

- * Can't handle complex, ambiguous, or new patterns.

- * Not scalable for large data.

2) Statistical approach(Machine Learning)

Language is treated like data, and algorithms learn patterns from it.

Use of models like:

Naive Bayes, Logistic Regression, SVM(Support Vector Machine)

Requires text to be converted into numbers using techniques like Bag of Words, TF-IDF, etc.

Example: Train a model on 10,000 product reviews. It learns which words indicate positive or negative sentiment and predicts for new reviews.

Used for:

Sentiment analysis, Spam detection, Text classification

Limitation:

* Loses context of words (like sarcasm or sequence of words)

Now what is SARCASM?

Yeah, because staying up all night debugging is exactly what I dreamed of.

ML Thinks: dreamed, exactly => sounds like passion

Reality: It's a painful dev life moment

I love it when the internet stops working during a meeting.

ML Thinks: love, meeting => positive

Reality: Sarcastic rage

3) Deep Learning-Based approach(Modern NLP)

Uses neural networks to automatically learn complex patterns and context from text.

- * Models can understand meaning, context, word order, and more.

- * Popular architectures;

 - RNN (Recurrent Neural Networks)

 - LSTM (Long Short Term Memory)

GRU (Gated Recurrent Units)

Transformers (BERT, GPT, T5, etc.)

Even in deep learning, we have to convert text into numbers but we don't use traditional techniques like Bag of Words or TF-IDF. Instead, we use more advanced methods like Word Embeddings (Words2Vec, GloVe) or TokenIDs with Embedding layers, which actually capture the meaning and context of words.

Now since we haven't learned Deep Learning yet, in this chapter we'll focus on how NLP works with traditional Machine Learning using techniques like Bag of Words and TF-IDF. Once we complete Deep Learning, we'll dive into advanced embedding techniques like Word2Vec and BERT, and build more powerful models using those.

Pipeline:

Now again pipeline are different for Machine learning and deep learning but still lets first see some of the first steps we do in both.

Step 1: Gathering the text or say getting the data.

- 1) You can use the pre build public data set for this.
- 2) You can do Web Scraping(When Data is Not available ready-made).
- 3) APIs (Structured + Legal Way to get live data)
- 4) Manual collection/crowdsourcing
- 5) Use NLP tools to auto generate data (Text Augmentation).

Step 2: Text Cleaning

Now, there are many things we do in Text cleaning, and depending on the task, let's see some examples

- 1) lowercasing; convert all text to lowercase.
- 2) Remove Punctuation; remove symbols like , ! ? @ # \$ % ^ & * () etc.

They don't add meaning in the ML context.

In deep learning-based NLP (especially with models like BERT, GPT, & T5), punctuation can carry meaning, and we usually keep it during preprocessing.

- 3) Remove Numbers; remove digits from the text (optional - depends on task).

I bought 5 phones => you may or may not need the number.

4) Remove URLs/links; remove any links in the text (very common in social media data).

5) Remove HTML tags; if scraping from web, remove things like <div>, <p>, etc.

6) Remove Emojis & Special Characters; emojis might break vectorization if not removed or handled.

7) Remove Stop words; remove common words like is, the, was, and, in, etc.

These don't help in understanding the meaning.

8) Spelling Correction (Optional): fix spelling mistakes using TextBlob or other libraries.

Corpus: A collection of texts. Think of it like a dataset of text.

Example: A folder of 10,000 tweets is your corpus. You can say; Text is one message. Corpus is the whole collection.

Sentence: A group of words that forms a meaningful statement.

Example: This is a spam message.

In NLP, some tools split long text into sentences before breaking them into words.

Token: Each individual word or symbol in the text is called a token.

Example: I love NLP => Tokens = [I, love, NLP]

The process of splitting text into tokens is tokenization.

Libraries:

Removing Stopwords: We use NLTK library, also spacy as well.

```
(import nltk)
```

```
(from nltk.corpus import stopwords)
(from nltk.tokenize import word_tokenize)
(nltk.download('punkt'))
(nltk.download('stopwords'))
```

How to know the english stopwrods?

```
stop_words = set(stopwords.words('english'))
len(stop_words)
```

Answer: 198

Step 3 : Feature Extraction / Vectorization:

One hot Encoding

Bag of Words

TF-IDF

Word2Vec

BERT/Transformer

The way we convert words into numbers in a smart way that's called feature extraction.

1) One hot encoding:

you must have word knowledge before starting

e.g. corpus

document is single piece of text e.g. a single line, para, article depends on your project

vocabulary in NLP is the unique words or dictionary the model understand converts words into number e.g "I love pasta", "I love pizza", ["I", "Love", "Pasta", "Pizza"]

Least used algo in moder NLP

Pros

i) It is intuitive

ii) It is easy to implement

Cons

- i) It is sparsity - too many zeros
- ii) Does not survive on too big corpus
- iii) OOV - Out Of Vocabulary => Vocabulary is limited
- iv) Size Difference => ML needs fixed sizes e.g. 4 input predict on that 4 input but here different 2x2 matrices different size
- v) Semantic meaning is not getting captured e.g. there are 3 words

Time Success iPad

Time 1 0 0 Success 0 1 0 iPad 0 0 1

3d space => if we plot a 3d graph they share same distance $\sqrt{2}$ but in real life they are not equally co-related

2) Bag of Words

Technique to convert text into numbers.

Works sentence wise not word wise

It matters only words frequency

max features means which word is repeated most time to keep it

Order doesnt matter in bag of words

Pros

i) Easy to implement

ii) Fixed size

iii) Now we can use in the ML model vector and document size remain fixed

iv) Work with ML models

v) Fast and efficient

Cons

i) Sparse matrix lots of zeros remains

ii) Out of vocabulary

iii) Semantic meaning (Better than OHE)

iv) Out of order

n-grams hyperparameter

Now you will select multiple words to make a vocabulary set

bigram, trigram, quadgram, ngram...

in **bigram** we see two words akash watch,
watch sheryans, harsh also, sheryans teach,
teach akash

ignore the remaining same pairs

like this similarly we make bigram, trigram, till
ngram...

trigram => akash watch sheryans, harsh also
sheryans.....

if we create same thing using unigram and
bigram dimensions change from 5d to 6d

n gram can be called as bag of ngrams

Pros

- i) Semantic meaning a little bit
- ii) same as bow benefits

Cons

- i) Dimensionality increase
- ii) out of vocabulary

3) TF-IDF

TF = term frequency(counting of each and every word)

IDF = inverse document frequency

TF*IDF

TF => No of occurrence of Term in Document /
Total Number of Terms in Documents => $1/3$
(Akarsh)

IDF $\Rightarrow \log_e(\text{Total number of documents in Corpus}) / (\text{Number of Documents with Term in them}) \Rightarrow \log_e 3 / 2$ (Akarsh)

For sheriyans $\Rightarrow$ IDF $\Rightarrow \log_e 3/3$

Idf calculates the rarity of word

Harsh $\Rightarrow \log_e 3/1$

Now to convert in matrix $\Rightarrow \text{TF} * \text{IDF} \Rightarrow \text{Akarsh}$
 $\Rightarrow 1/3 * \log_e(3/2) \Rightarrow 0.05869708635$

Pros

i) Information retrieval e.g. Google Search

Cons

i) Sparsity on large documents many 0's

ii) Out of Vocabulary if we give a new word so it will not exist based on that word

iii) Dimensions